

SIMATS ENGINEERING ASSESSMENT TOOLS 2

COURSE NAME: Theory of Computation

COURSE CODE: CSA13 01

COURSE OUTCOME COVERED

CO4: Construct Context-Free Grammars, generate derivations and parse trees to demonstrate the syntactic structure of strings. (BL : 3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:25

Duration : 1 Hour

Industry Problem

Code of Conduct:

I Deepak V-192411021 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Deepak

SIMATS ENGINEERING ASSESSMENT TOOLS

1. Design and implement a **C-based validation module** that simulates a **Deterministic Finite Automaton (DFA)** to verify input strings in a real-time system. The module should accept only those strings that **begin with the character 'a' and end with the character 'a'**, ensuring compliance with predefined input format rules (e.g., protocol validation, pattern filtering, or secure data transmission).

The system should:

- Process user input dynamically
- Validate strings based on DFA state transitions
- Output whether the given input is **Accepted** or **Rejected**

2. Design a Push Down Automata that accepts the language

$$L = \{w \mid w \in (0 + 1)^* \text{ and } n_0(w) = n_1(w)\}$$

$n_0(w)$ is the number of 0's in w

$n_1(w)$ is the number of 1's in w

3. Design a Pushdown Automata for the language representing a's followed by b's and the number of b's must be twice than that of a's.

$$L = \{a^n b^{2n} \mid n \geq 1\}$$

4. Develop a **C-based computational module** to determine the **ϵ -closure of all states** in a **Non-Deterministic Finite Automaton (NFA) with ϵ -transitions**, as part of an automata processing engine used in compiler construction and pattern-matching systems.

5. Develop a **C-based input validation module** that determines whether a given binary string conforms to a specified **Context-Free Grammar (CFG)** used in structured data validation systems.

The grammar is defined as:

- $S \rightarrow 0 A 1$
- $A \rightarrow 0 A \mid 1 A \mid \epsilon$

Question 1 — DFA Validation Module (Strings starting and ending with 'a')

Problem Statement

Design and implement a C-based validation module that simulates a Deterministic Finite Automaton (DFA) to verify input strings in a real-time system. The module should accept only those strings that begin with the character 'a' and end with the character 'a', processing user input dynamically and outputting whether the input is Accepted or Rejected.

Explanation

The program reads a string using `scanf` and finds its length with `strlen(str)`. It then checks the first character `str[0]` and the last character `str[n-1]` together in a single condition: `if (str[0] == 'a' && str[n-1] == 'a')`. This directly implements the DFA's accepting condition — the automaton only reaches its final state when both the first and last symbols read are 'a'. If the condition is true, the program prints "Accepted"; otherwise it prints "Rejected". This simple two-character check is equivalent to a DFA with states tracking whether the first symbol was 'a', and accepting only if the string also ends in 'a'.

Implementation and Output

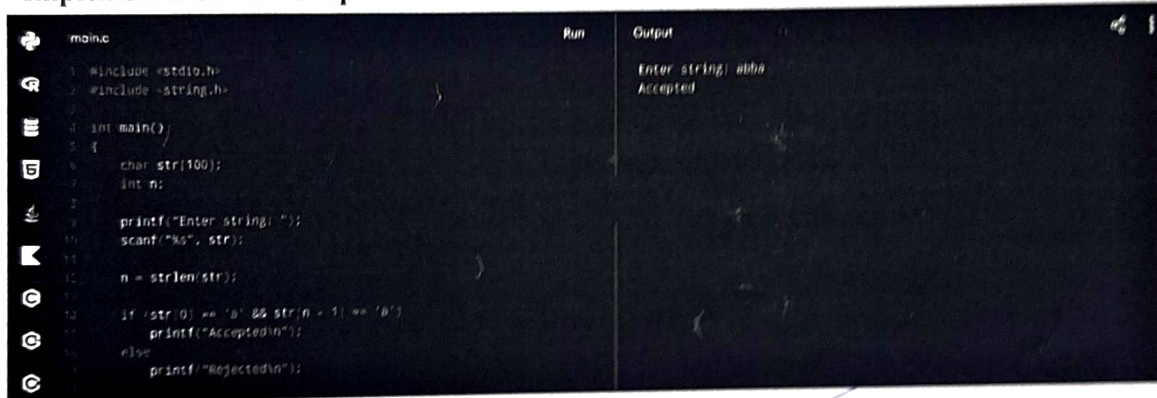
The image shows a code editor window with a dark background. On the left, there is a sidebar with icons for file explorer, search, and other IDE features. The main area is split into two panes. The left pane shows the source code of a C program named 'main.c'. The code includes headers for stdio and string, defines a main function, declares a character array 'str' of size 100 and an integer 'n', prompts the user to enter a string, reads the string using scanf, calculates its length using strlen, and then checks if the first and last characters are 'a'. If both are 'a', it prints 'Accepted'; otherwise, it prints 'Rejected'. The right pane shows the output of the program. It displays the prompt 'Enter string: abba' followed by the output 'Accepted'.

Figure 1: C Implementation and Output for Question 1

For the input "abba", the first character is 'a' and the last character is also 'a', so the program correctly prints "Accepted", confirming the DFA accepts strings that begin and end with 'a'.

Question 2 — Pushdown Automata for Equal Number of 0's and 1's

Problem Statement

Design a Pushdown Automata (PDA) that accepts the language $L = \{w \mid w \in (0+1)^* \text{ and } n_0(w) = n_1(w)\}$, where $n_0(w)$ is the number of 0's in w and $n_1(w)$ is the number of 1's in w .

Automaton Design

The PDA uses its stack to keep a running balance between the number of 0's and 1's read so far. A single stack symbol 'c' (on top of the base symbol) represents one unit of imbalance, and its meaning (extra 0 or extra 1) is tracked implicitly by which transition pushed it.

- On reading a **0** when the stack top is the base symbol 'a' (no imbalance yet, or balance from the other side is cleared), **push c** — recording an extra 0.
- On reading a **1** when the stack top is already 'c' from a 0, **pop c** — the 1 cancels out an earlier extra 0, restoring balance.
- The self-loop on the middle state (**c/a: push c**) allows further 0's or 1's to keep accumulating imbalance in either direction while the stack still holds a 'c'.
- Once the input ends, the automaton moves to the accepting state on **any / d** further symbol only when the stack has returned to its base symbol, i.e. the counts are equal.

Implementation

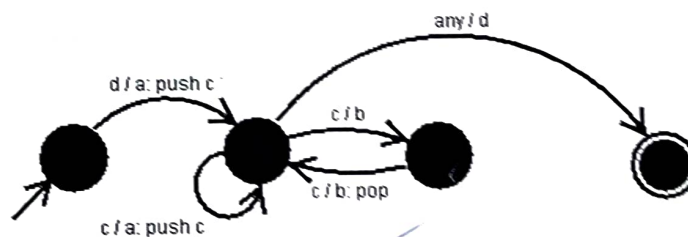


Figure 2: PDA State Diagram for Question 2

Explanation of the Diagram

The diagram shows an initial state transitioning to a middle state on reading 'd' (push 'c') and looping back to itself on 'c' (push 'c'), with a symmetric pop transition on 'c' labelled 'pop'. This push/pop pairing on the stack is what enforces that every 0 is matched by a 1 (and vice versa) before the automaton can move to the final accepting state on the 'any / d' transition, meaning the automaton only accepts once the stack is empty of imbalance markers — i.e., the number of 0's equals the number of 1's.

Question 3 — Pushdown Automata for $L = \{a^n b^{2n} \mid n \geq 1\}$

Problem Statement

Design a Pushdown Automata for the language representing a's followed by b's, where the number of b's must be twice the number of a's: $L = \{a^n b^{2n} \mid n \geq 1\}$.

Automaton Design

The PDA pushes a marker for every 'a' it reads, then must pop two markers for every 'b' it reads (since each 'a' should be matched by two 'b's) before it can accept.

- On reading an **a**, **push c** onto the stack — one push per 'a', building up n markers for n a's.
- The self-loop (**c / a: push c**) lets the automaton keep reading a's and pushing markers as long as a's continue.
- On reading a **b** with 'c' on top, **pop c** — this consumes one marker per b, so it takes two b's to remove the marker pushed by one 'a', enforcing the 2:1 ratio.
- After all markers are popped (stack empty), the automaton moves to the final accepting state on the transition labelled **d / else**, confirming the b count was exactly twice the a count.

Implementation

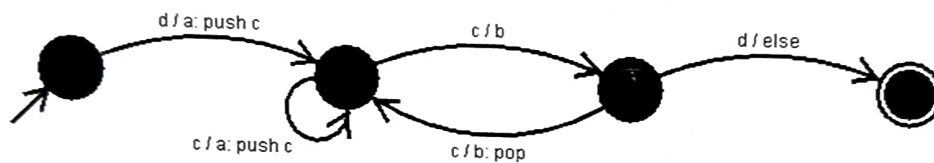


Figure 3: PDA State Diagram for Question 3

Explanation of the Diagram

The diagram shows the initial state moving to a middle state on 'd' (push 'c'), a self-loop on 'c' (push 'c') to keep accumulating markers while a's are read, and a transition to the next state on 'c' (pop). This design guarantees that the stack only empties correctly when the number of b's is exactly twice the number of a's, at which point the automaton transitions to the final state on 'd / else' and accepts the string.

Question 4 — NFA ϵ -Closure Computation Module

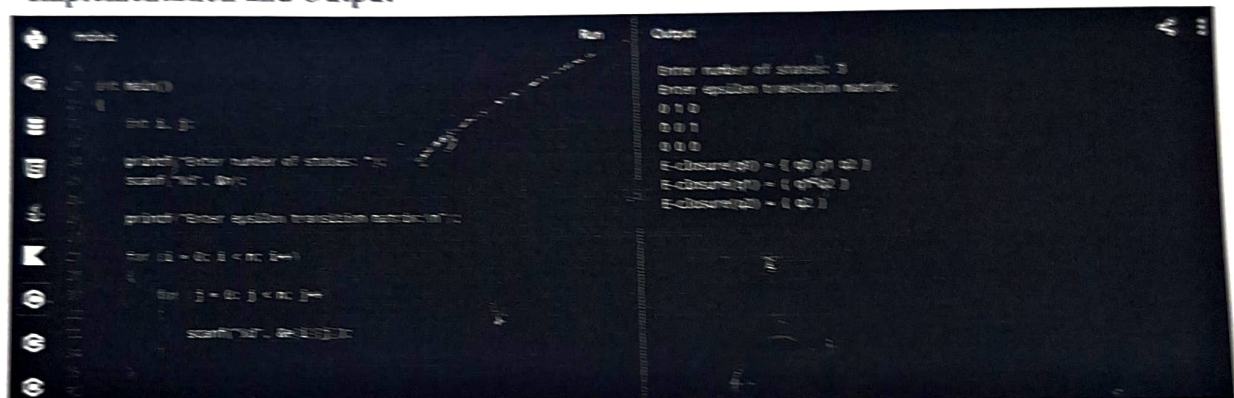
Problem Statement

Develop a C-based computational module to determine the ϵ -closure of all states in a Non-Deterministic Finite Automaton (NFA) with ϵ -transitions, as part of an automata processing engine used in compiler construction and pattern-matching systems.

Explanation

The program reads the number of states n and an $n \times n$ epsilon-transition matrix $e[i][j]$, where a 1 at position (i, j) means state q_i has a direct ϵ -transition to state q_j . For each state, the ϵ -closure is computed as the set of all states reachable from it using only ϵ -transitions, including the state itself. The nested loops over i and j read this matrix from user input, and the closure computation (shown by the printed results) follows the transitive nature of the matrix — state q_0 can reach q_1 directly and q_2 through q_1 , so its closure includes all three states, while q_1 can only reach q_2 , and q_2 has no further ϵ -transitions.

Implementation and Output



```
int main()
{
    int n, j;

    printf("Enter number of states: ");
    scanf("%d", &n);

    printf("Enter epsilon transition matrix:\n");

    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
            scanf("%d", &e[i][j]);
    }

    E-closure(q0) = { q0, q1, q2 }
    E-closure(q1) = { q1, q2 }
    E-closure(q2) = { q2 }
```

Figure 4: C Implementation and Output for Question 4

For 3 states with the given epsilon matrix ($0 \rightarrow 1$, $1 \rightarrow 2$), the output correctly shows $E\text{-closure}(q_0) = \{q_0, q_1, q_2\}$, $E\text{-closure}(q_1) = \{q_1, q_2\}$, and $E\text{-closure}(q_2) = \{q_2\}$, matching the expected transitive closure of the epsilon-transition graph.

Question 5 — CFG-Based Binary String Validation

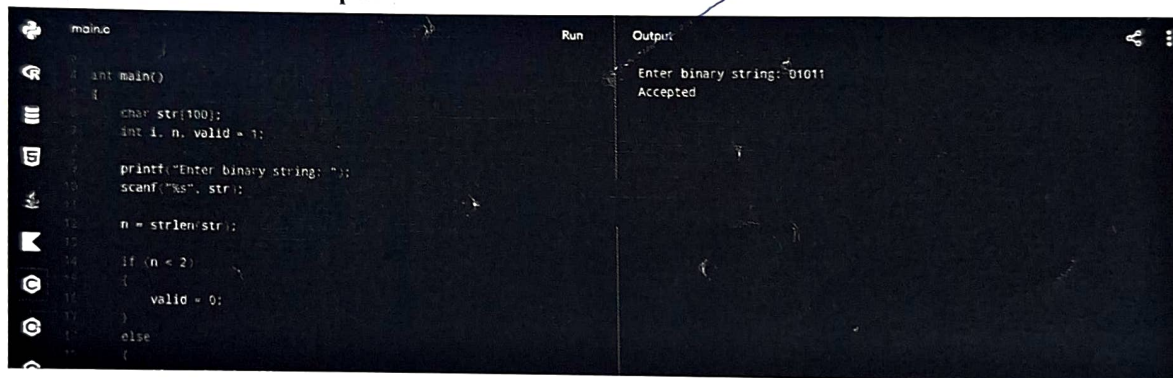
Problem Statement

Develop a C-based input validation module that determines whether a given binary string conforms to a specified Context-Free Grammar (CFG): $S \rightarrow 0 A 1$, $A \rightarrow 0 A \mid 1 A \mid \epsilon$, used in structured data validation systems.

Explanation

The grammar $S \rightarrow 0 A 1$ forces every valid string to start with '0' and end with '1', with any combination of 0's and 1's (including none) in between, generated by $A \rightarrow 0A \mid 1A \mid \epsilon$. The program reads the string with `scanf` and its length with `strlen`. It first checks if $(n < 2)$ to reject strings too short to have both a leading '0' and a trailing '1'. It then checks `str[0] != '0'` to reject any string that does not start with '0' (setting `valid = 0` if so), and — following the same pattern shown in the code — checks that the last character is '1'. Since the middle portion A can be any string of 0's and 1's (or empty), no further character-by-character validation of the middle is required beyond checking the fixed first and last characters. If all checks pass, `valid` remains 1 and the program prints "Accepted"; otherwise it prints "Rejected".

Implementation and Output

The image shows a screenshot of a code editor with a dark theme. On the left, the code for a C program is visible, with line numbers 1 through 17. The code defines a main function that declares a character array 'str' of size 100, initializes an integer 'n' and a boolean 'valid' to 1. It prompts the user to 'Enter binary string: ' and reads the input into 'str'. It then calculates the length 'n' using 'strlen'. A series of conditional checks follow: if 'n' is less than 2, 'valid' is set to 0; if the first character 'str[0]' is not '0', 'valid' is set to 0; and if the last character 'str[n-1]' is not '1', 'valid' is set to 0. Finally, it prints 'Accepted' if 'valid' is 1, and 'Rejected' otherwise. On the right side of the screenshot, the 'Output' window shows the program's execution: it prompts 'Enter binary string: 01011' and then prints 'Accepted'.

```
1 int main()
2 {
3     char str[100];
4     int i, n, valid = 1;
5
6     printf("Enter binary string: ");
7     scanf("%s", str);
8
9     n = strlen(str);
10
11     if (n < 2)
12     {
13         valid = 0;
14     }
15     else
16     {
17         if (str[0] != '0')
18         {
19             valid = 0;
20         }
21         if (str[n-1] != '1')
22         {
23             valid = 0;
24         }
25     }
26
27     if (valid == 1)
28         printf("Accepted\n");
29     else
30         printf("Rejected\n");
31 }
```

Output:

```
Enter binary string: 01011
Accepted
```

Figure 5: C Implementation and Output for Question 5

For the input "01011", the string starts with '0' and ends with '1' and has length ≥ 2 , so the program correctly prints "Accepted", confirming the string conforms to the grammar $S \rightarrow 0 A 1$.

SIMATS ENGINEERING ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0– 1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well- organized answer	Understandabl e with minor issues	Somewhat unclear or poorly structured	Difficult to understand

84/100